

Step 3 — Filing a Claim: `submitClaim` (EIP-1967, EIP-1822)

Alice's flight is cancelled. She files a claim through the frontend. This step is also where it's worth pausing to explain a fact true of *every* contract call in this whole system, not just this one: none of these contracts are called at their own "logic" address.

EIP-1967 and EIP-1822 — true for every call in this entire trace

InsurancePolicy, ClaimRegistry, and OracleAdapter are all deployed as **UUPS upgradeable proxies**. The addresses in this document's address book (0x2222...bbbb , 0x3333...cccc , 0x4444...dddd) are ERC1967Proxy addresses — thin contracts that hold *all* the real storage and forward every call to whichever separate "implementation" contract currently holds the actual logic.

- **EIP-1967** standardizes *where*, in storage, a proxy tracks its current implementation address — at the fixed slot `bytes32(uint256(keccak256("eip1967.proxy.implementation")) - 1)`, deliberately not slot `0`, so it can never collide with a contract's own declared state variables. This is what lets tooling (Etherscan, `forge`, `ethers`) reliably answer "what code does this proxy currently run" without any contract-specific knowledge.
- **EIP-1822 (UUPS)** is *where the upgrade authority lives*: instead of a separate, permanently-deployed admin contract holding upgrade power (the older "Transparent Proxy" pattern), the `upgradeToAndCall` function lives on the **implementation contract itself**, gated in this project by each contract's own `_authorizeUpgrade`, which requires `UPGRADER_ROLE` (the Timelock).

Concretely, for the call this document traces (`ClaimRegistry.submitClaim`):

```
Alice's wallet
-> sends a transaction to 0x3333...cccc (the ClaimRegistry PROXY)
-> ERC1967Proxy.fallback() reads the implementation address from
    its EIP-1967 storage slot
-> delegatecall's into that implementation contract's code
    -> the code executes submitClaim's logic, but reads and writes
        the PROXY's storage (0x3333...cccc's own storage) - the
        implementation contract's own storage is never touched by a
        real transaction
<- return data flows back through the proxy
<- Alice's wallet receives the result (a claim ID, or a revert)
```

This is worth internalizing once here because it applies silently to every other call in this whole document set — every mention of "ClaimRegistry.submitClaim" or "InsurancePolicy.getPolicy" is really this same delegatecall pattern happening underneath.

The actual sequence

1. Frontend calls `ClaimRegistry.submitClaim`

- **Contract:** ClaimRegistry at 0x33333333333333333333333333333333CCCC
- **Function:** submitClaim(uint256 policyId, bytes32 merkleRoot, uint256 amount)
- **Input:** policyId = 1, merkleRoot = 0xdeadbeef... (a placeholder hash of Alice's claim description — real evidence hashing isn't built yet, see this project's own "still open" notes), amount = 221 ether
- **Imported contracts:** the locally-declared IInsurancePolicyRegistry interface (structurally mirrors InsurancePolicy's Policy struct shape without a full import — see this project's explanatory documentation for why), IERC20 / SafeERC20 (used later,

in Step 5, not this function)

- **Traversal, in order:**

1. Checks `221 ether <= maxPayoutPerClaim` (configured high enough at deployment) — passes.
2. Checks `policyRegistry != address(0)` — it's `0x2222...bbbb`.
3. Calls `IInsurancePolicyRegistry(0x2222...bbbb).getPolicy(1)` — an external **STATICCALL** (read-only, no reentrancy risk) into `InsurancePolicy` (step 2 below).
4. Receives back the `Policy` struct written in Step 2: `{ policyId: 1, coverageAmount: 10_000 ether, holder: 0x1111...aaaa, validFrom: 1_800_000_000, validUntil: 1_831_536_000, revoked: false, metadataHash: 0xabc... }`.
5. Checks `revoked == false`, `block.timestamp` (say `1_800_500_000`) is within `[1_800_000_000, 1_831_536_000]`, and `holder == msg.sender` → `0x1111...aaaa == 0x1111...aaaa` — all pass.
6. Calls `IInsurancePolicyRegistry(0x2222...bbbb).isPremiumCurrent(1)` — another **STATICCALL** (step 3 below).
7. Receives `true` (Alice paid in Step 2; `premiumPaidUntil[1] = 1_802_592_000`, comfortably ahead of `block.timestamp = 1_800_500_000`).
8. Calls the internal `_consumedCoverage(1)`: loops `policyClaims[1]` (currently empty — this is Alice's first claim on this policy), returns `0`.
9. Checks `0 + 221 ether <= 10_000 ether` — plenty of headroom, passes.
10. Assigns `claimId = nextClaimId (1)`. Stores into `claims[1]`:

```
Claim {
  claimId: 1, policyId: 1, claimant: 0x1111...aaaa,
  amount: 221 ether, status: Submitted,
  submittedAt: 1_800_500_000, processedAt: 0,
  merkleRoot: 0xdeadbeef...
}
```

- ```
11. Pushes 1 into policyClaims[1].
12. Emits ClaimSubmitted(1, 1, 0x1111...aaaa, 0xdeadbeef..., 221 ether,
 1 800 500 000).
```

- **Output:** uint256 claimId = 1.

## 2. InsurancePolicy.getPolicy (called from step 1.3)

- **Contract:** InsurancePolicy at 0x222222222222222222222222222222BBB
- **Function:** getPolicy(uint256 policyId) external view returns (Policy memory)
- **Input:** policyId = 1
- **Traversal:** checks \_exists[1] == true; returns \_policies[1] verbatim — pure read, no state change (this is *why* it compiles to a STATICCALL from ClaimRegistry's side).
- **Output:** the Policy struct shown above.

### 3. InsurancePolicy.isPremiumCurrent (called from step 1.6)

- **Function:** `isPremiumCurrent(uint256 policyId) public view returns (bool)`
- **Input:** `policyId = 1`
- **Traversal:** `premiumTerms[1].amountPerPeriod != 0` (it's 33 ether, billing is active for this policy — not exempt); returns `block.timestamp (1_800_500_000) <= premiumPaidUntil[1] (1_802_592_000) + premiumGracePeriodSeconds`.
- **Output:** `true`.

## Value trace summary

```
claims[1].claimant: 0x11111111111111111111111111111111AAAA
claims[1].amount: 221 ether (2210000000000000000000)
```

```
claims[1].status: Submitted
policyClaims[1]: [1] (this claim's ID, appended)
```

No token has moved yet in this step — `submitClaim` only records intent. Money moves in Step 5, and only if Step 4's verification approves it.